User Study

Instructions for Grading. **PLEASE READ CAREFULLY**.

Overview

Imagine you are a user interacting with a chatbot service (e.g., Gemini, ChatGPT, or DeepSeek) for help with programming tasks. On each page, you will see:

- **A Question**: This could be a vague user query or a code snippet requiring clarification or completion. Questions containing code will be displayed in code blocks for clarity.
- **Two Responses**: labeled Answer A and Answer B

Your Task

Please evaluate Answer A and Answer B based on the following criteria:

- **Precision and Focus**: How directly the response targets the question's core issue, using clear guidance, examples, or code.
- **Contextual Fit**: How well the response matches the user's programming goal (e.g., debugging, feature implementation).
- **Answer Faithfulness**: How effectively the response provides actionable steps or code to achieve the user's desired outcome.
- **Correctness**: Correctness in LLM responses refers to the accuracy and factual validity of the information, code, or guidance provided, ensuring it is free from errors and aligns with the user's query and intended outcome.

How to Rate

For each question, you will provide two types of ratings:

1. **Preference Rating (1–5 scale):**
    - **"1"**: Answer A is better, based on the criteria above.
    - **"5"**: Answer B is better, based on the criteria above.
    - **"3"**: Both answers are equally good, based on the criteria above.

Note:

- **Incomplete Responses**: Some responses may appear cut off due to a 256-word limit imposed on the chatbots. Please grade only the content provided, ignoring any abrupt endings or non-English text (e.g., stray characters).
- **Code Formatting**: Questions and responses containing code will be presented in code blocks or artifact tags for clarity. Ensure you review these carefully for correctness and relevance.
- **Consistency**: Try to apply the same standards across all evaluations to ensure fair comparisons.

Question 1

```python
def send_email(to, subject, body):
    import smtplib
    server = smtplib.SMTP('smtp.example.com')
    # TODO: implement email sending
```

Answer A:

```python
import smtplib
from email.mime.text import MIMEText

def send_email(to, subject, body):
    """
    Sends an email.

    Args:
        to: The recipient's email address.
        subject: The subject of the email.
        body: The body of the email.
    """
    try:
        server = smtplib.SMTP('smtp.example.com')  # Replace with
your SMTP server
        server.starttls()  # Secure the connection
        server.login('your_username', 'your_password')  # Replace
with your credentials
        server.sendmail(to, [to], body)
        server.quit()
        print("Email sent successfully!")
    except Exception as e:
        print(f"Error sending email: {e}")
    finally:
        if 'server' in locals():
```

```
            server.quit()  # Close the connection


if __name__ == '__main__':
    # Example usage
    to = 'recipient@example.com'
    subject = 'Hello from my script!'
    body = 'This is a test email sent using Python
```

Answer B:

"***Extra context***:

User Question:

```
Python
      def send_email(to, subject, body):
      import smtplib server = smtplib.SMTP('smtp.example.com')
      # TODO: implement email sending
```

AI clarification question: *What specific functionality are you hoping to achieve with this email sending functionality? For example, do you want to send a simple email with a specific recipient, subject, and body, or are you looking to implement features like authentication, email formatting, or handling?*

User follow-up response: *I want to send a simple email with authentication using Gmail's SMTP server."*

AI Final Answer:

Okay, that's a great starting point! You want to send a simple email using Gmail's SMTP server. Here's a breakdown of the specific functionality you'll want to achieve and how to approach it, along with code examples. I'll focus on the core aspects you've requested.

1. Core Functionality: Sending an Email

The `send_email` function should take the recipient's email address (`to`), the email subject (`subject`), and the email body (`body`) as input. It then uses the `smtplib` library to connect to Gmail's SMTP server and send the email.

2. Implementation Steps and Code Example

Python

```python
import smtplib
from email.mime.text import MIMEText

def send_email(to, subject, body):
    """
    Sends a simple email with authentication using Gmail's SMTP
server.

    Args:
        to: The recipient's email address.
        subject: The email subject.
        body: The email body.
    """
    try:
        # Create the email message
        msg = MIMEText(body)  # Create a text-based email message
        msg['Subject'] = subject
        msg['From'] = 'your_email@gmail.com'  # Replace with your
email
        msg['To'] = to

        # Connect to Gmail SMTP server and send email
        server = smtplib.SMTP('smtp.gmail.com', 587)
        server.starttls()  # Secure the connection
        server.login('your_email@gmail.com', 'your_password')  #
Replace with your credentials
        server.send_message(msg)
        server.quit()
        print("Email sent successfully!")
    except Exception as e:
```

```
        print(f"Error sending email: {e}")
```

***How to rate**:*

"1": strongly prefer answer A
"5": strongly prefer answer B
"3": both answers are equally good

Precision and Focus (1-5): 4
Contextual Fit (1-5): 4
Answer Faithfulness (1-5): 4
Correctness (1-5): 3

Question 2

I need help with my user interface.

Answer A:

"***Extra context***:

User Question: *I need help with my user interface.*
AI clarification question: *What kind of user interface are you working on? (e.g., website, mobile app, desktop application, or something else?) And what specific issue are you facing?*
User follow up response: *I'm building a React website, and the buttons aren't responding to clicks."*

AI Final answer:

Okay, let's tackle this! A React website with unresponsive buttons is a common and frustrating problem. It's great you're looking for help – we can definitely diagnose and fix it.
To help me understand the situation better and give you the most targeted advice, I need a little more information. Let's start with these questions:
1. Let's talk about the buttons themselves:

- What kind of buttons are they? (e.g., `<button>`, `<a>`, `<div>` with button class)
- Where are they located on the page?
- Can you show me the code for the buttons?
- Are they styled with CSS?

2. What's happening exactly when you click them?

- Do you see any error messages in the browser console?
- Are the clicks not registering at all, or is there some other behavior?

With this info, I can help you debug the problem effectively.

Answer B:

I understand you need help with your user interface. Could you please provide more details about the platform or framework you're using? For example, is this a website, mobile app, or desktop application? Also, what specific issues are you facing with the user interface? The more details you provide, the better I can assist you.

> ***How to rate***:
> ***"1"***: strongly prefer answer A
> ***"5"***: strongly prefer answer B
> ***"3"***: both answers are equally good

Precision and Focus (1-5): 2
Contextual Fit (1-5): 2
Answer Faithfulness (1-5): 2
Correctness (1-5): 2

Question 3

```python
def read_file(file_path):
    with open(file_path, 'r') as file:
        # TODO: Read and process file content
```

Answer A:

```python
def read_file(file_path):
    with open(file_path, 'r') as file:
        content = file.read()
    return content
```

Answer B:

"***Extra context***:
User Question:

```python
    def read_file(file_path):
    with open(file_path, 'r') as file:
    # TODO: Read and process file content
```

AI clarification question: *What type of content are you expecting the file to contain (e.g., text, binary, CSV)? Specifically, what processing tasks do you intend to perform on the file content after reading it?*
User follow up response: *It's a CSV file, and I want to parse it into a list of dictionaries.*"

AI Final answer:
Great! Since your file is a CSV and you want to parse it into a list of dictionaries, here is an improved version of your `read_file` function with error handling:

Python

```python
import csv

def read_file(file_path):
    """
    Reads a CSV file and parses it into a list of dictionaries.
    Args:
        file_path (str): The path to the CSV file.
    Returns:
        list: List of dictionaries representing CSV rows.
    """
    data = []
    try:
        with open(file_path, 'r', newline='') as csvfile:
            reader = csv.DictReader(csvfile)
            for row in reader:
                data.append(row)
    except FileNotFoundError:
        print(f"File not found: {file_path}")
    except Exception as e:
        print(f"An error occurred: {e}")
    return data
```

This reads the CSV file assuming it has a header row and returns the data as a list of dictionaries.

***How to rate**:*
"1": strongly prefer answer A
"5": strongly prefer answer B
"3": both answers are equally good

Precision and Focus (1-5): 5
Contextual Fit (1-5): 5
Answer Faithfulness (1-5): 5
Correctness (1-5): 5

Question 4

```JavaScript
var myObject = {
    name: 'ChatGPT',
    // TODO: Add properties
};

console.log(myObject);
```

Answer A:

```JavaScript
var myObject = {
    name: 'ChatGPT',
};

console.log(myObject);
```

Answer B:

*"**Extra context**:*
***User Question**:*

```JavaScript
var myObject = {
name: 'ChatGPT',
// TODO: Add properties
 };
console.log(myObject);
```

***AI clarification question**: What context are you using for this object? (e.g., are you working with Node.js? What functionality do you want to achieve?)*
***User follow up response**: I'm using Node.js, and I want to add an `age` property to store a number."*

AI Final answer:
Great! To add an `age` property to your object in Node.js, you can define it like this:

```JavaScript
var myObject = {
    name: 'ChatGPT',
    age: 36 // Adding an age property
};

console.log(myObject);
```

This will output:

```JSON
{ name: 'ChatGPT', age: 36 }
```

Let me know if you want help adding more properties or methods.

> ***How to rate**:*
> ***"1"**: strongly prefer answer A*
> ***"5"**: strongly prefer answer B*
> ***"3"**: both answers are equally good*

Precision and Focus (1-5): 5
Contextual Fit (1-5): 5
Answer Faithfulness (1-5): 5
Correctness (1-5): 5

Question 5

```JavaScript
function fetchData(url) {
    // TODO: Implement fetch logic
    return;
}

fetchData('http://example.com')
```

Answer A:
"***Extra context***:
User Question:

```JavaScript
function fetchData(url) {
    // TODO: Implement fetch logic
    return;
}
fetchData('http://example.com')
```

AI clarification question: *What is the intended purpose of the* `fetchData` *function, and what data or action should the function perform after receiving a URL? For example, do you want to fetch data from a specific API, or something else entirely?*
User follow up response: *I want to fetch JSON data from a REST API and return it."*

AI Final answer:
You've correctly identified the purpose!
The `fetchData` function is designed to make an HTTP GET request to a specified URL and retrieve JSON data. Here's a typical implementation using the Fetch API:

```JavaScript
function fetchData(url) {
  return fetch(url)
    .then(response => {
```

```
      if (!response.ok) {
        throw new Error(`HTTP error! Status:
${response.status}`);
      }
      return response.json();
    })
    .then(data => {
      console.log(data);
      return data;
    })
    .catch(error => {
      console.error('Error fetching data:', error);
      throw error;
    });
}

fetchData('http://example.com');
```

Answer B:

JavaScript

```
function fetchData(url) {
  // TODO: Implement fetch logic
  fetch(url)
    .then(response => {
      if (!response.ok) {
        throw new Error(`HTTP error! Status:
${response.status}`);
      }
      return response.json(); // Assuming the URL returns JSON
    })
```

```
    .then(data => {
      console.log(data); // Log the fetched data
    })
    .catch(error => {
      console.error('Error fetching data:', error);
    });
}

fetchData('http://example.com')
```

***How to rate**:*

***"1"**: strongly prefer answer A*
***"5"**: strongly prefer answer B*
***"3"**: both answers are equally good*

Precision and Focus (1-5): 1
Contextual Fit (1-5): 2
Answer Faithfulness (1-5): 2
Correctness (1-5): 2